

Proyecto Funcional

Procesamiento Paralelo de Imágenes con Scheme y Erlang

Eddy Ramírez Jiménez

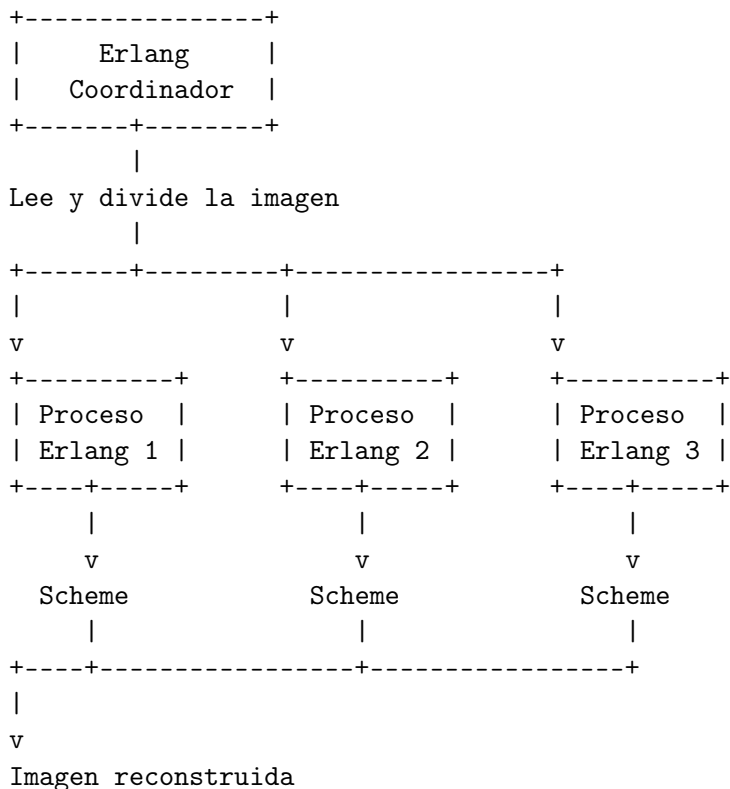
27 de agosto de 2026

1. Descripción general

El objetivo del proyecto es desarrollar un sistema para el procesamiento paralelo de imágenes que integre los lenguajes de programación funcional **Scheme** y **Erlang**.

El sistema deberá utilizar Erlang como lenguaje encargado de la concurrencia y coordinación del procesamiento, mientras que Scheme será responsable de realizar las transformaciones sobre las imágenes.

La arquitectura general del sistema será:



Cada proceso Erlang deberá encargarse de una región de la imagen y deberá ejecutar una instancia independiente de Scheme para procesarla.

2. Objetivos

Al finalizar el proyecto, el estudiante deberá demostrar dominio de los siguientes conceptos:

- Programación funcional mediante Scheme.

- Manipulación recursiva de estructuras de datos.
- Representación y procesamiento de imágenes.
- Creación y comunicación entre procesos Erlang.
- Paralelización de tareas.
- Comunicación entre programas escritos en diferentes lenguajes.
- Diseño de protocolos de comunicación.
- Medición y análisis del rendimiento de programas paralelos.

3. Especificación del sistema

El sistema recibirá como entrada una imagen y una cantidad de procesos concurrentes. La imagen deberá ser procesada mediante un filtro implementado en Scheme.

Como mínimo, el sistema deberá implementar un **difuminado gaussiano (Gaussian Blur)**. La ejecución deberá seguir, conceptualmente, las siguientes etapas:

1. Erlang recibe el nombre de la imagen de entrada.
2. Erlang lee la imagen.
3. Erlang divide la imagen en regiones.
4. Erlang crea los procesos encargados del procesamiento.
5. Cada proceso recibe una región de la imagen.
6. Cada proceso ejecuta una instancia independiente de Scheme.
7. Scheme procesa la región utilizando el filtro correspondiente.
8. Scheme devuelve la región procesada.
9. Erlang recibe los resultados.
10. Erlang reconstruye la imagen original a partir de las regiones procesadas.
11. Erlang genera la imagen de salida.

4. Formato de imagen

Para evitar que el proyecto dependa de bibliotecas externas de procesamiento de imágenes, se deberá utilizar el formato **PPM**.

Se recomienda utilizar el formato P3, cuya estructura básica es:

```
P3
800 600
255
255 0 0
255 0 0
...
```

El programa deberá ser capaz de leer y escribir correctamente imágenes PPM. Cada píxel deberá poder representarse mediante sus componentes:

$$(R, G, B)$$

donde cada componente representa un valor entre 0 y 255.

5. Parte I: procesamiento en Scheme

Scheme será responsable exclusivamente del procesamiento de una región de la imagen.

El programa en Scheme deberá implementar, como mínimo, una función equivalente a:

`(gaussian image-region kernel)`

La función deberá recibir una región de la imagen y un kernel de convolución y deberá producir una nueva región con el filtro aplicado.

Como mínimo, deberá soportarse un kernel gaussiano de tamaño 3×3 .

Por ejemplo:

$$G = \frac{1}{16} \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}.$$

Para cada píxel, el filtro deberá calcular una combinación ponderada de los píxeles vecinos.

El procesamiento deberá realizarse utilizando las características propias de Scheme. Se espera el uso apropiado de funciones, recursión, listas y abstracciones funcionales.

5.1. Restricciones sobre Scheme

Scheme no deberá encargarse de:

- Crear los procesos paralelos.
- Distribuir el trabajo.
- Coordinar los procesos.
- Reconstruir la imagen completa.

Estas responsabilidades corresponden a Erlang.

6. Parte II: procesamiento en Erlang

Erlang será el responsable de coordinar toda la ejecución.

Deberá implementar un proceso coordinador que:

1. Lea la imagen de entrada.
2. Determine sus dimensiones.
3. Divida la imagen en regiones.
4. Cree los procesos necesarios.
5. Distribuya las regiones entre los procesos.
6. Ejecute Scheme para cada región.
7. Reciba los resultados.
8. Reconstruya la imagen.
9. Escriba la imagen resultante.

El número de procesos deberá poder especificarse como parámetro de ejecución.

Por ejemplo:

`./image_processor imagen.ppm salida.ppm 4`

indicaría que la imagen deberá procesarse utilizando cuatro procesos Erlang.

7. Comunicación entre Erlang y Scheme

Uno de los objetivos fundamentales del proyecto es integrar dos lenguajes diferentes.

Cada proceso Erlang deberá ejecutar una instancia independiente del programa Scheme.

El mecanismo concreto de comunicación entre ambos lenguajes deberá ser diseñado e implementado por los estudiantes.

El protocolo deberá permitir, como mínimo:

- Enviar una región de imagen a Scheme.
- Especificar los parámetros del filtro.
- Indicar el tamaño del kernel.
- Recibir la región procesada.
- Detectar errores durante la ejecución de Scheme.

El informe deberá explicar claramente el protocolo utilizado y justificar las decisiones de diseño.

8. Procesamiento de regiones

La imagen deberá dividirse en regiones para permitir el procesamiento concurrente.

Por ejemplo, una imagen podría dividirse de la siguiente manera:

+-----+		
	R1	
+-----+		
	R3	
+-----+		

El método exacto de división queda a criterio del estudiante.

Sin embargo, los filtros de convolución presentan un problema importante: el valor de un píxel puede depender de píxeles ubicados en regiones vecinas.

Por ejemplo, para un kernel 3×3 , un píxel cercano al límite entre dos regiones puede necesitar información de la región contigua.

Por esta razón, el sistema deberá implementar un mecanismo de **halo** o región de contexto.

Cada proceso podrá recibir píxeles adicionales alrededor de su región, pero deberá devolver únicamente los píxeles correspondientes a la región que le fue asignada.

9. Procesamiento de los bordes

El sistema deberá especificar qué sucede cuando el kernel requiere píxeles que se encuentran fuera de la imagen.

Se permitirá utilizar cualquiera de las siguientes estrategias:

- Extender el borde.
- Utilizar píxeles con valor cero.

- Repetir el píxel más cercano.
- Reflejar la imagen.

La estrategia seleccionada deberá estar documentada y ser utilizada consistentemente.

10. Concurrency

La solución deberá utilizar el modelo de concurrencia de Erlang.

Cada región deberá ser procesada por un proceso Erlang independiente.

La comunicación entre procesos deberá realizarse mediante el mecanismo de paso de mensajes de Erlang.

No se permitirá implementar la paralelización mediante una simple secuencia de llamadas.

La solución deberá demostrar que las regiones pueden procesarse de forma concurrente.

11. Tolerancia a fallos

El sistema deberá ser capaz de detectar el fallo de un proceso encargado de procesar una región.

Como mínimo, deberá detectarse cuando una instancia de Scheme termine incorrectamente.

El sistema deberá informar el error y evitar que una falla silenciosa produzca una imagen incorrecta.

Como extensión, se recomienda implementar la posibilidad de volver a ejecutar automáticamente una región cuyo procesamiento haya fallado.

12. Modos de ejecución

El sistema deberá permitir comparar al menos dos configuraciones:

1. Ejecución utilizando un único proceso.
2. Ejecución utilizando múltiples procesos.

Se recomienda realizar experimentos con:

1, 2, 4, 8

procesos, cuando el hardware disponible lo permita.

Para cada configuración deberán registrarse los tiempos de ejecución.

13. Evaluación del rendimiento

El informe deberá analizar el comportamiento del sistema utilizando como mínimo las siguientes métricas.

13.1. Tiempo de ejecución

Para cada cantidad de procesos p , se deberá medir:

$$T_p$$

correspondiente al tiempo necesario para procesar la imagen completa.

13.2. Speedup

Se deberá calcular:

$$S_p = \frac{T_1}{T_p}.$$

13.3. Eficiencia

También deberá calcularse:

$$E_p = \frac{S_p}{p}.$$

Los resultados deberán presentarse mediante tablas y gráficos.

El análisis deberá explicar las razones por las cuales el speedup obtenido puede ser menor que el speedup ideal.

Entre los aspectos que deberán considerarse se encuentran:

- Creación de procesos.
- Comunicación entre procesos.
- Comunicación entre Erlang y Scheme.
- Lectura y escritura de archivos.
- División y reconstrucción de la imagen.
- Copia y serialización de datos.

14. Extensiones

Además del filtro gaussiano obligatorio, se deberá implementar al menos **una transformación adicional**.

Algunas posibilidades son:

- Escala de grises.
- Inversión de colores.
- Ajuste de brillo.
- Threshold.
- Sharpen.
- Detección de bordes mediante Sobel.

La transformación adicional deberá ser implementada en Scheme.

Como extensión adicional, se podrá permitir que el usuario especifique una secuencia de transformaciones.

Por ejemplo:

```
(pipeline
  (grayscale)
  (gaussian 3)
  (threshold 128))
```

15. Interfaz mínima

El sistema deberá proporcionar una interfaz que permita ejecutar al menos:

```
./image_processor entrada.ppm salida.ppm 1
./image_processor entrada.ppm salida.ppm 2
./image_processor entrada.ppm salida.ppm 4
./image_processor entrada.ppm salida.ppm 8
```

El último parámetro representa la cantidad de procesos Erlang que deberán utilizarse. La interfaz podrá modificarse siempre que conserve la funcionalidad solicitada.

16. Entregables

El proyecto deberá incluir:

1. Imágenes de prueba

Un conjunto de imágenes utilizadas para verificar el funcionamiento del sistema.

2. Manual de ejecución

Instrucciones suficientes para compilar, configurar y ejecutar el proyecto.

3. Informe técnico

El informe deberá explicar:

- Arquitectura del sistema.
- División de la imagen.
- Modelo de concurrencia utilizado.
- Comunicación entre Erlang y Scheme.
- Protocolo de comunicación.
- Implementación del filtro.
- Manejo de los bordes.
- Manejo de errores.
- Resultados experimentales.
- Análisis del speedup y eficiencia.
- Limitaciones del sistema.

4. Demostración

El proyecto deberá ser presentado y ejecutado durante la demostración correspondiente.

17. Restricciones

- El procesamiento del filtro obligatorio deberá realizarse en Scheme.
- La creación y coordinación de los procesos deberá realizarse en Erlang.
- Cada región deberá ser procesada mediante una instancia independiente de Scheme ejecutada desde Erlang.
- No se permite realizar todo el procesamiento de la imagen en un único proceso Scheme y utilizar Erlang únicamente como lanzador.

- No se permite utilizar una biblioteca externa que implemente directamente el filtro gaussiano.
- La solución deberá poder ejecutarse utilizando las versiones de Erlang y Scheme indicadas por el profesor.

18. Criterios de evaluación

La calificación se distribuirá de la siguiente manera:

Componente	Porcentaje
Implementación en Scheme	20 %
Implementación en Erlang	25 %
Integración Erlang-Scheme	20 %
Concurrencia y distribución	15 %
Procesamiento correcto de imágenes	10 %
Informe y análisis experimental	10 %
Total	100 %

19. Defensa del proyecto

Durante la defensa, los estudiantes deberán ser capaces de explicar cualquier parte del código desarrollado.

El evaluador podrá solicitar modificaciones pequeñas al programa durante la defensa, por ejemplo:

- Cambiar el tamaño del kernel.
- Modificar la cantidad de procesos.
- Cambiar la estrategia para los bordes.
- Incorporar un nuevo filtro.
- Forzar el fallo de un proceso.
- Modificar la forma de dividir la imagen.

La defensa deberá demostrar que los integrantes comprenden tanto la implementación en Scheme como la implementación en Erlang.

La fecha de entrega será el día 18 de setiembre.